Structured Multi-Agent Systems for Autonomous Development Operations

A Design Whitepaper: Architecture, Principles, and Implementation Guidance

Version 1.0 · May 2026

Stack-agnostic · Reproducible from first principles

ABSTRACT

This paper describes the architecture, design principles, and operational patterns of a four-layer multi-agent system for autonomous software development. The system coordinates specialized agents through a hierarchical orchestration model, maintains cross-session context through a tiered hybrid retrieval store, and enforces accountability through an append-only audit mechanism and a pre-response reasoning contract shared by all agents. The design is stack-agnostic: the concepts are grounded in peer-reviewed research on multi-agent coordination, retrieval-augmented generation, and human-in-the-loop oversight, and can be reproduced in any technology stack. The goal is to provide sufficient architectural depth that a practitioner could implement an equivalent system from scratch.

Keywords: multi-agent systems, retrieval-augmented generation, human-in-the-loop, audit trails, domain specialization, orchestration, autonomous software development, tiered memory

Contents

1Introduction and Motivation

The deployment of large language models (LLMs) as autonomous agents in software development contexts has progressed rapidly from single-agent assistants toward coordinated multi-agent systems capable of executing multi-phase engineering operations with minimal human intervention. Early adopters observed a consistent limitation: individual agents, regardless of capability, accumulate context debt across sessions, make unverifiable claims, and lack any structural mechanism preventing destructive or irreversible operations. The result is a system that performs impressively in short bursts and degrades unpredictably over time.

This paper describes an architecture that addresses those failure modes directly. The design is organized around four principles: **domain isolation** (each agent has exactly one domain and a hard escalation path when requests exceed it); **structural accountability** (all agents share a pre-response reasoning scan that makes uncertainty, contradictions, and undocumented implementations visible in the output); **durable memory** (a tiered retrieval store ensures that decisions made in one session are retrievable in the next, and that safety invariants structurally cannot be outranked by recency); and **human-gated irreversibility** (destructive operations require explicit human confirmation before execution).

The architecture is described in stack-agnostic terms. No specific framework, language model provider, database, or operating environment is prescribed. Specific implementations are cited where they illustrate a design choice, but every component described here can be built with a range of available technologies. The intent is that a practitioner reading this paper should be able to implement an equivalent system from first principles.

1.1 Problem Statement

Autonomous agent systems operating in software development contexts face three structural failure modes that ad-hoc tooling consistently fails to address.

- **Context degradation.** LLM context windows are finite and do not persist between sessions. Without an external memory store, every session starts from zero. Agents rediscover the same facts, repeat the same mistakes, and cannot build on prior decisions. Lewis et al. established the foundational pattern for addressing this through retrieval-augmented generation (RAG), which combines parametric model knowledge with non-parametric external retrieval.[1]
- **Unverifiable claims.** Agents operating near the edge of their knowledge produce confident-sounding output regardless of actual certainty. Without a structural mechanism that surfaces uncertainty in-band, downstream agents and human reviewers cannot distinguish grounded claims from plausible confabulation.
- **Unconstrained irreversibility.** As agents gain tool-use and execution capabilities, their actions acquire real-world consequences. A misinterpreted instruction can delete files, drop database records, or misconfigure infrastructure before any human has the opportunity to intervene. Research on human-in-the-loop (HITL) systems identifies pre-execution confirmation gates as the primary mitigation for this class of failure.[2]

1.2 Scope

This paper covers: the four-layer system architecture; the design of the orchestration, specialist, memory, and knowledge layers; the structured output protocol that makes inter-agent communication parseable; the audit contract that governs pre-response reasoning; governance mechanisms including human-in-the-loop gates; and implementation guidance sufficient to reproduce the system. Related work from the multi-agent systems, RAG, and AI safety literatures is integrated throughout.

2System Overview: Four Architectural Layers

The system is organized into four layers, each with a distinct function, distinct components, and explicit interfaces to adjacent layers. The layers are not deployment tiers (they may run on the same hardware) — they are logical boundaries that enforce separation of concerns and prevent cross-layer contamination of responsibilities.

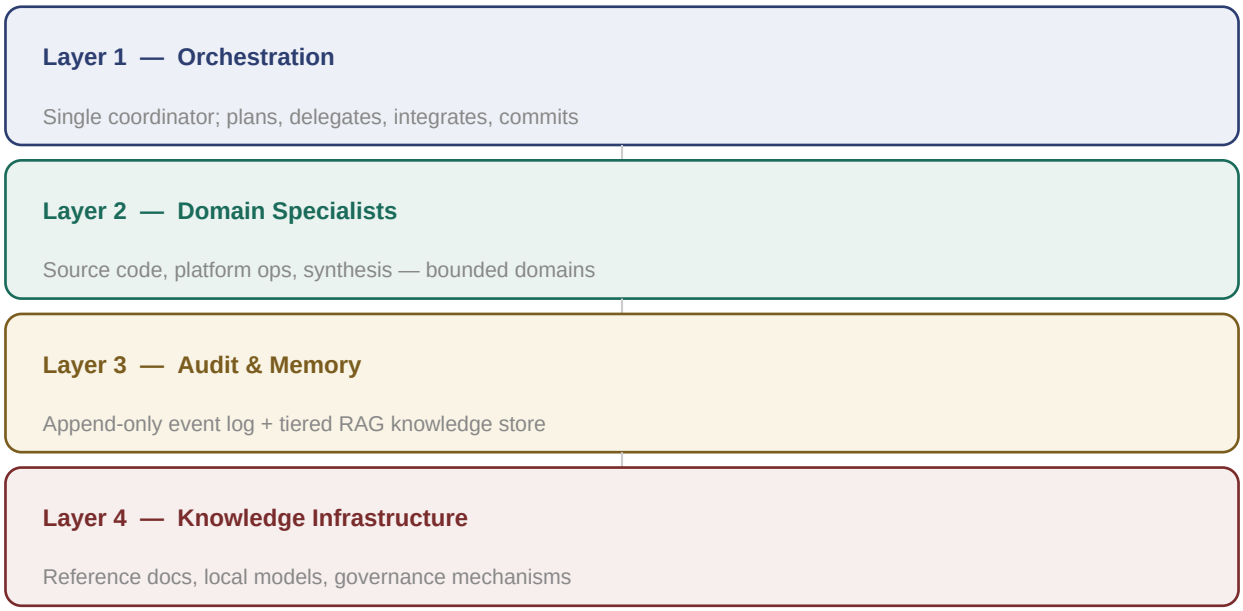

Figure 1. Four-layer architecture. Each layer has a defined function, a bounded interface, and an explicit escalation path when a request exceeds scope.

Layer	Function	Primary output	Key constraint
1 — Orchestration	Plans, delegates, integrates, commits	Delegation briefs; commit records	Never executes directly
2 — Specialists	Domain-bounded technical execution	Structured result blocks	One domain per agent
3 — Audit & Memory	Event recording + context retrieval	Audit records; context packs	Write-only / read-only separation
4 — Knowledge	Docs, models, governance	Retrieved context; invariants	Canonical rules never expire

Table 1. Layer summary.

A key design decision is that the layers are defined by function, not by the number of agents. Layer 2 (specialists) may contain two agents or ten; what matters is that each specialist has exactly one domain,

one output format, and one escalation path. This structure is consistent with findings in the multi-agent systems literature that role-specialized heterogeneous systems outperform homogeneous multi-agent architectures on complex tasks requiring complementary capabilities.[3]

3Layer 1: Orchestration

The orchestration layer contains a single coordinator agent. Its function is to receive task requests, decompose them into specialist delegations, sequence those delegations, integrate the structured results, and commit finalized work to the project state (version control, documentation, changelogs). The coordinator does not directly modify source code, infrastructure files, or knowledge store contents — those actions are delegated to the appropriate specialist.

This centralized coordination model is consistent with the architecture of frameworks such as AutoGen (Wu et al., 2024), which demonstrated that a central orchestrator managing conversable specialist agents significantly outperforms single-agent approaches on multi-step engineering tasks, particularly when tasks require cross-domain expertise.[4] The key distinction in the design described here is that the coordinator's authority is bounded: it can plan and integrate, but it cannot act on any domain it does not own.

3.1 Responsibilities

- **Sprint planning.** Decompose multi-phase work into discrete delegation briefs, each specifying task mode, goal, relevant paths, constraints, and the expected output format.
- **Sequencing.** Determine delegation order. Some specializations have ordering dependencies (e.g., infrastructure must be established before application code is tested against it). The coordinator holds and enforces this ordering.
- **Integration.** Receive structured result blocks from specialists, reconcile conflicts, detect orphaned findings, and produce a unified output that is consistent across all domains.
- **Audit and compliance.** Run the shared audit contract before every response. Detect documentation drift (implementations that close tracked items without updating tracking documents). Package SOP obligations and execute them before committing.
- **Commit authority.** The coordinator is the only agent with authority to commit, push, or otherwise finalize work. This creates a single, auditable commit path.

3.2 What the Coordinator Must Not Do

Hard boundary	The coordinator must not directly modify source code, infrastructure files, entitlement definitions, or any other artifact owned by a domain specialist. If the coordinator finds itself reasoning about how to implement a Rust function or a PowerShell registry write, that is a signal to issue a delegation brief, not to proceed. Crossing this boundary collapses the domain isolation that makes the system auditable.

This constraint is enforced at the protocol level: the coordinator's output format includes delegation blocks addressed to named specialists, not implementation artifacts. Any implementation artifact appearing in coordinator output should be treated as an architecture violation.

3.3 Delegation Brief Format

A delegation brief is a structured message from the coordinator to a specialist. Minimum required fields: **task mode** (plan, exec, debug, or audit); **goal** (one-sentence description); **relevant paths** (files, services, or directories in scope); **constraints** (destructive operations permitted? Elevation required?); and **context** (reference to any prior plan being executed). The specialist responds with a structured result block whose schema is defined in Section 7.

4Layer 2: Domain Specialists

Domain specialist agents perform focused technical work within hard domain boundaries. Each specialist owns exactly one domain — a defined set of file types, system layers, or technical responsibilities — and has an explicit escalation path for requests that exceed that domain. Specialists never commit directly; they return structured result blocks to the coordinator.

The design reflects the principle of bounded contexts from domain-driven design and the microservices principle that each component should have minimal, well-defined responsibilities.[5] In multi-agent system research, this pattern is described as heterogeneous specialization: agents with complementary, non-overlapping capabilities consistently outperform both single-agent systems and homogeneous multi-agent configurations on tasks requiring diverse expertise.[3]

4.1 Specialist Archetypes

The following archetypes describe the functional roles that specialist agents fill. A given implementation may combine or further subdivide these roles depending on system scale and task complexity.

Source Code Specialist(s)

Owns application source code in a specific language or framework. Handles feature implementation, bug fixes, refactoring, and code-level diagnostics. Does not touch infrastructure, build pipelines, or files belonging to other source code specialists. In systems with multiple languages or frameworks (e.g., a Rust backend and a TypeScript frontend), each language gets its own specialist with a hard file-extension boundary.

DOMAIN CONSTRAINTS:

- Domain: named file types and directories only
- Escalation: scope block if change requires crossing language boundary
- Never commits — returns code artifact blocks to coordinator

Platform / Infrastructure Specialist(s)

Owns the runtime environment: OS-layer scripting, service management, build pipelines, containerization, dependency management, code signing, packaging, and system-level diagnostics. In cross-platform systems, a separate specialist per target OS is the recommended pattern. Platform specialists are plan-first agents: they produce a reviewable plan before executing any side-effecting command.

DOMAIN CONSTRAINTS:

- Domain: OS layer, build tooling, service management
- Per-OS specialization recommended for cross-platform targets
- Execution Gate: plan → CONFIRM → execute (see Section 9)

Synthesis / Analysis Specialist

Reads recorded events, session state, and knowledge store contents to produce health assessments, compliance scores, and structured findings. This agent is read-only with respect to all persistent state — it never writes to the audit database, the knowledge store, or source files. Its function is analysis and synthesis, not execution.

DOMAIN CONSTRAINTS:

- Domain: read-only access to audit and memory layers
- Output: health assessments, compliance scores, finding summaries
- No write access — ever

Flow / Automation Specialist

Handles multi-step automation flows that span multiple systems or require coordination beyond what a single specialist can execute. Distinct from the coordinator: the flow specialist executes a bounded automation task, while the coordinator manages the overall sprint plan and commit sequence.

DOMAIN CONSTRAINTS:

- Domain: cross-system automation flows with defined start/end state
- Returns FLO_PLAN and FLO_RESULT blocks
- Execution Gate applies to all side-effecting steps

4.2 Scope Escalation

When a specialist detects that completing a task would require modifying artifacts outside its domain, it must not attempt to proceed. Instead it returns a scope escalation block to the coordinator, describing what unexpected scope was detected and recommending how the coordinator should re-brief or involve additional specialists. This mechanism is critical for maintaining domain boundaries under pressure — specialists should be designed to resist the temptation to 'helpfully' make a small change outside their domain, because such changes bypass the accountability mechanisms of the specialist that owns that domain.

4.3 The Principle of Least Privilege Applied to Agents

Domain boundaries in this architecture are a direct application of the Principle of Least Privilege (PoLP) to agent design. Traditional PoLP requires that every process have only the minimum access rights needed for its specific function.[6] Applied to agents: each specialist has read/write access only to the artifacts it owns. A source code specialist has no authority over infrastructure files; a platform specialist has no authority over application source. This limits the blast radius of any individual agent error or misinterpretation to the scope of that agent's domain.

5Layer 3: Audit and Memory

The audit and memory layer consists of two systems with complementary but strictly separate functions. The **event recorder** answers the question: *what happened, and when?* The **knowledge store** answers the question: *what do we know?* These questions require different storage models, different retrieval semantics, and different governance rules. Combining them into a single system forces compromises that make both worse. Keeping them separate allows each to be optimal for its own purpose.

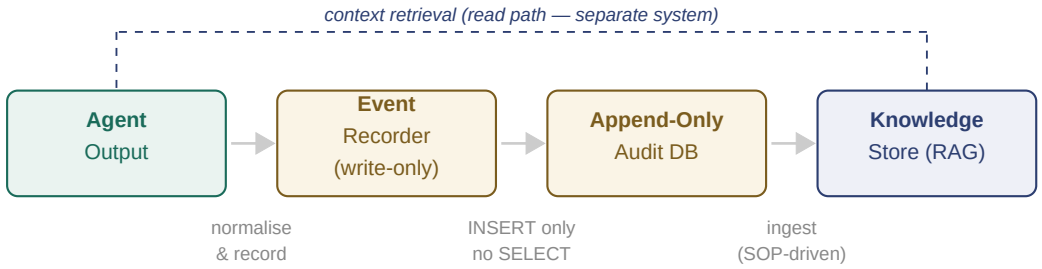

Figure 2. Dual recording architecture. Agent output flows into the event recorder (write-only path). The knowledge store is populated separately via a session lifecycle SOP (read path shown dashed). The two systems share no data path during normal operation.

5.1 The Event Recorder (Append-Only Audit Trail)

The event recorder is an append-only write surface that normalizes and persists every significant agent output as a structured record. Its design constraints are non-negotiable: no UPDATE operations, no DELETE operations, no read path from the recorder itself. The recorder writes; a separate synthesis agent reads independently.

This separation of write and read surfaces is directly motivated by research on auditable agentic systems. Kacianka and Pretschner (2021) demonstrate that accountability in multi-agent settings requires tamper-resistant logging as a substrate, and that agents should be assigned 'meta-goals' of maintaining accurate records alongside their primary task goals.[7] The write-only constraint ensures the recorder cannot be influenced by the agents it records.

Core properties:

Property	Requirement	Rationale
Append-only	INSERT only; no UPDATE, no DELETE	Immutable audit trail; past events cannot be revised

Write-only surface	Recorder exposes one tool: record()	Prevents recorder from being queried to shape agent behavior
Normalization	Raw output normalized to structured records	Enables structured queries by synthesis agent
WAL mode	Write-Ahead Logging on backing store	Concurrent reads and writes; crash recovery
Flag extraction	Inline audit markers extracted pre-normalization	Preserves audit signal regardless of normalization outcome

Table 2. Event recorder core properties.

5.2 Normalization and Structured Records

Raw agent output is normalized into a structured record schema before storage. Normalization is performed by a lightweight local model (a small quantized LLM running locally) operating at temperature zero for deterministic output. The normalization process: extracts a one- to two-sentence summary; identifies atomic observations; assigns an action type from a controlled vocabulary; extracts topic tags; and assigns a confidence rating. Critically, the normalization model is instructed to record only what is explicitly stated, making no inferences beyond the raw input.

The action type vocabulary is closed and versioned. Recommended base vocabulary includes: session open/close, code change, command result, test result, error, decision, finding, delegation dispatch, delegation outcome, and observation only. The vocabulary can be extended for domain-specific event types, but extensions should be additive only — removing or redefining existing types breaks historical queries.

5.3 The Synthesis Agent

A synthesis agent reads the audit database directly and produces session health assessments, compliance scores, and structured findings. It is read-only with respect to all persistent state. The synthesis agent never passes data through the recorder — it reads the database directly, independently. This is the dividual pair pattern: one agent writes, one agent reads, and they never communicate with each other. The database is the trust boundary.

Design principle

The synthesis agent's independence from the recorder is not just an implementation detail — it is a correctness guarantee. If the recorder could influence what the synthesis agent reads, the audit trail could be shaped to present a favorable view of system behavior. The read and write surfaces must remain separate for the audit trail to be trustworthy.

5.4 The Knowledge Store: Tiered RAG

The knowledge store provides cross-session context continuity through a tiered retrieval-augmented generation (RAG) architecture. The foundational approach was established by Lewis et al. (2020), who

demonstrated that combining parametric model memory with non-parametric external retrieval significantly improves performance on knowledge-intensive tasks while reducing hallucination.[1]

The architecture described here extends the base RAG pattern with two key additions: tiered document classification with deterministic priority scoring, and a hybrid retrieval mechanism combining dense vector search with BM25 sparse retrieval.

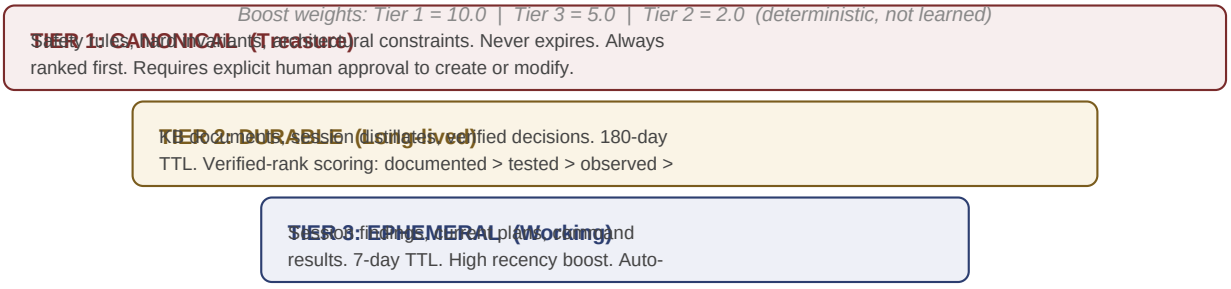

Figure 3. Three-tier memory architecture. Width represents relative priority. Tier 1 (Canonical) always surfaces first regardless of recency. Boost weights are fixed constants, not learned parameters.

5.5 Tier Design

Tier 1 (Canonical / Treasure). Safety rules, architectural invariants, and non-negotiable constraints. Never expires. Always included first in any context pack. Requires explicit human approval (confirmation gate) to create or modify. The tier boost constant (10.0 in the reference implementation) is a fixed value, not a learned weight — this ensures canonical content structurally cannot be outranked by recent working notes regardless of semantic similarity. An incorrect canonical entry is more dangerous than a missing one; the confirmation gate reflects this asymmetry.

Tier 2 (Long-lived / Durable). Knowledge base documents, distilled session summaries, and verified architectural decisions. Default TTL of 180 days. A verification rank differentiates document quality within the tier: documented > tested > observed > asserted. Two documents at equal semantic distance score differently based on how their claims were established. This is particularly important for architectural decisions, where an asserted claim (an agent's inference) should rank below a documented claim (explicitly recorded by a human).

Tier 3 (Working / Ephemeral). Session findings, current task plans, command results, and in-progress decisions. Default TTL of 7 days. High recency boost. Automatically expires and is not retained. Promotion to Tier 2 occurs through an explicit compaction operation that distills a session's working records into a single long-lived summary.

5.6 Hybrid Retrieval: Dense + Sparse

Pure vector (dense) retrieval excels at semantic similarity but misses exact-term matches; pure keyword (sparse) retrieval captures exact terminology but lacks semantic understanding. Research on hybrid

retrieval systems consistently shows that combining the two approaches outperforms either alone, with improvements in recall of up to 580% on challenging retrieval tasks compared to dense-only baselines.[8]

The recommended hybrid approach: generate a dense vector embedding of each document chunk using a local embedding model (768-dimensional embeddings from a model such as nomic-embed-text provide a good balance of quality and computational cost); maintain an FTS5 full-text search index over the same chunks using BM25 ranking; combine the scores with a deterministic weighting scheme augmented by the tier boost constants; and apply token-budget-aware context packing to produce the final context pack delivered to the requesting agent.

The backing store can be SQLite with FTS5 extension. SQLite's embedded, zero-infrastructure model makes it suitable for IDE-adjacent deployments where running a dedicated vector database process would add operational complexity without proportional benefit at the scale of hundreds to low thousands of document chunks. At larger scales (approximately 10,000+ chunks), approximate nearest-neighbor indexes (HNSW, IVF) should be evaluated.

5.7 Context Packing and Token Budget

The retrieval output must be assembled into a context pack that fits within the token budget of the requesting agent. The recommended packing strategy: always include all Tier 1 results first (they are safety-critical and must not be crowd out by volume); then fill the remaining budget with Tier 3 (high recency) and Tier 2 results, ranked by combined score. Deduplicate by content hash before packing. A reference budget of 1,400 tokens for the context pack leaves sufficient room for the agent's instructions and output within typical LLM context windows.

6Layer 4: Knowledge Infrastructure

The knowledge infrastructure layer provides the persistent assets that all other layers depend on: the reference document corpus, the local inference models, and the governance mechanisms that enforce system-wide behavioral constraints.

6.1 Reference Document Corpus

Each domain specialist should have a set of first-party reference documents ingested into the knowledge store as Tier 2 (Long-lived) entries. These documents serve as the grounding layer for specialist claims: when a specialist asserts a fact about a system API, command-line flag, or protocol behavior, that claim should be traceable to a specific document in the knowledge store. Specialists that cannot ground a claim in an ingested document or a verified external source should emit a grounding-gap audit marker rather than assert the claim confidently.

For platform/infrastructure specialists in a cross-platform system, the recommended corpus includes: shell and scripting language references; service management documentation; build tooling references; container and packaging documentation; signing and distribution documentation; and runtime configuration guides for any AI inference infrastructure. The exact documents will vary by technology stack; the structural requirement is that each specialist has a defined set of authoritative references that cover its domain.

Implementation note	Documents should be ingested with an idempotent update mechanism: if a document is re-ingested with unchanged content (verified by content hash), the operation should be a no-op. This allows documents to be re-ingested safely as part of the session lifecycle SOP without creating duplicate entries.

6.2 Local Inference Models

Where possible, AI inference should be performed locally rather than through external API calls. This recommendation is driven by three concerns: latency (local inference eliminates network round-trips for tasks that run in a tight session loop); cost (session-boundary normalization and embedding operations may run hundreds of times per day; external API costs at this volume are significant); and data sovereignty (session records and knowledge store contents may contain sensitive project information that should not leave the local environment).

Three local model roles are recommended: a **normalization model** (small, fast, runs at temperature zero — a 3B parameter quantized model is sufficient); a **synthesis model** (larger, higher quality — 7B to 14B parameters, used for session health assessment and structured analysis); and an **embedding model** (purpose-built for dense retrieval — 768-dimensional sentence transformer models such as nomic-embed-text provide good retrieval quality at reasonable computational cost). The orchestrator and specialist roles should use the best available reasoning model, local or cloud-based, depending on

capability requirements and data sensitivity constraints.

6.3 Governance Mechanisms

Governance mechanisms in this layer define the behavioral constraints that all agents must respect. Three mechanisms are described here; each is implemented at the agent protocol level, not merely as documentation.

- **The audit contract** (detailed in Section 8) is a pre-response reasoning requirement shared by all agents. It is implemented as a mandatory scan of every planned response before output.
- **The execution gate** (detailed in Section 9) requires explicit human confirmation before any destructive or irreversible operation.
- **The canonical entry governance gate** requires explicit human confirmation to create or modify Tier 1 (Canonical) knowledge store entries. This gate exists because incorrect canonical entries — safety rules, architectural invariants — are more dangerous than missing ones. The confirmation friction is intentional.

7Inter-Layer Communication and Structured Output Contracts

One of the most important design decisions in a multi-agent system is the format of inter-agent communication. Systems that use free-form natural language for agent-to-agent communication introduce ambiguity, make programmatic parsing unreliable, and produce integration surfaces that are difficult to test. The architecture described here uses **structured output contracts**: every agent produces machine-parseable output blocks with a defined schema, and every coordinator-to-specialist message is a delegation brief with a defined minimum field set.

This approach is consistent with the SOP-driven communication patterns of frameworks like MetaGPT (Hong et al., 2023), which demonstrated that structured communication protocols between specialized agents significantly reduce coordination errors compared to unconstrained agent conversations.[9]

7.1 Output Block Schema

Each specialist defines a set of named output blocks. All blocks share a common structure: a start delimiter, a task identifier, required fields, optional fields, and an end delimiter. The start and end delimiters allow reliable extraction of structured content from a response that also contains natural language prose.

Block type	When emitted	Required fields
PLAN	Before any execution	task_id, goal, steps (with reversibility ratings), requires_elevation
RESULT	After execution completes	task_id, outcome, steps_completed, output_summary, follow_up_required
DEBUG	After failure diagnosis	error_source, root_cause, evidence, fix_applied
AUDIT	After audit task	scope, findings (id, category, severity, what, why, how)
SCOPE_ESCALATION	Domain boundary exceeded	task_id, reason, unexpected_steps, recommendation
SOP_HANDOFF	SOP obligation triggered	trigger, reason, files_changed, suggested_summary

Table 3. Specialist output block types and required fields.

7.2 Reversibility Rating

Every step within a PLAN block carries a reversibility rating. This rating drives the human confirmation gate (Section 9) and provides the coordinator with the information needed to assess risk before authorizing execution. Three levels are defined: **safe** (fully reversible or read-only); **recoverable** (reversible with effort, e.g., file moves with backup, service restarts); **destructive** (not reversible without external backup, e.g., delete, drop, hard reset). Agents must never downgrade a 'destructive' rating to 'recoverable' to reduce friction. The rating reflects reality, not convenience.

7.3 Recursion Guard

Agents invoked in background (coordinator-delegated) mode must not invoke other agents during the same invocation. Background mode is depth-1 only. This prevents unbounded delegation chains that would make the coordinator's integration pass impossible and audit trail reconstruction unreliable. If a background-mode agent determines that its task requires capabilities from another specialist, it returns a scope escalation block to the coordinator, which can then sequence a new delegation. The coordinator owns delegation depth; specialists do not.

8The Audit Contract: Pre-Response Reasoning

The audit contract is a mandatory pre-response reasoning scan executed by every agent before producing any output. It is the most structurally important behavioral constraint in the system, because it is the mechanism that makes agent uncertainty, self-contradiction, and untracked implementations visible in the output itself, rather than hidden in post-hoc logs.

The design is motivated by research on reasoning provenance in autonomous systems. Vispute (2026) identifies structured behavioral analytics — as opposed to mere execution traces — as the appropriate mechanism for understanding agent reasoning at population scale in production environments.[10] The audit contract operationalizes this by requiring agents to externalize specific reasoning states as inline markers.

Figure 4. The six audit contract markers. Each marker fires when a specific condition is detected in the agent's planned response. Markers are emitted inline, immediately before the affected content.

8.1 Marker Definitions

[[AUDIT:APPROACH_SWITCH]]

The agent is abandoning or meaningfully changing a stated approach from earlier in the session. Fires when an agent switches strategy without explicit user instruction. Enables downstream review of why a prior approach was abandoned.

[[AUDIT:OUTPUT_SUPERSEDES]]

The agent's output invalidates or replaces a prior plan, finding, or command sequence from the same session. Fires when a new plan makes a prior plan obsolete. Prevents two valid-looking plans from coexisting in the session record.

[[AUDIT:GROUNDING_GAP]]

The agent is making a claim that cannot be grounded in provided context, session history, knowledge store documents, or a whitelisted reference. This is the most important marker for controlling confabulation: it surfaces unverifiable claims rather than asserting them confidently.

[[AUDIT:CONTRADICTION]]

The agent's output conflicts with something established earlier in the session — a prior plan, stated constraint, or confirmed system fact. Fires when an agent produces output that is logically inconsistent with prior session state.

[[AUDIT:MODEL_UNCERTAINTY]]

The agent's confidence in this output is materially lower than baseline — it is uncertain but proceeding. Distinct from GROUNDING_GAP: GROUNDING_GAP fires when a source cannot be found; MODEL_UNCERTAINTY fires when a source exists but the agent is uncertain about its applicability or correctness.

[[AUDIT:DOC_DRIFT]]

The agent is producing an implementation artifact that closes a tracked checklist item or resolves a tracked stub, but no corresponding update to the tracking document (epic tracker, stub tracker, changelog) is in scope of this response. Fires to prevent implementations from outrunning documentation. This marker is exempt from the per-response cap — it fires once per untracked artifact.

8.2 Rate Controls and Integrity Rules

The audit contract includes rate controls to prevent noise from over-flagging: a maximum of three markers per response (with DOC_DRIFT exempt from this cap), and no repeated markers for the same condition within a single session exchange. Markers must flag material events only — not every uncertainty, routine command step, or expected inference. The rate controls reflect a signal quality tradeoff: an agent that emits an audit marker on every response has effectively made audit markers meaningless. Missing a marker that should have fired is an audit failure; over-flagging routine steps is noise. Both degrade the system.

A critical integrity rule: the audit contract is load-bearing identity, not an injected rule. This means the contract must be part of each agent's core behavioral specification, not added as a post-hoc instruction that the agent could reason around or deprioritize under pressure. The distinction matters: an agent that views the audit contract as an external constraint will find edge cases to justify not firing it; an agent for which the contract is part of its identity will apply it consistently.

9Human-in-the-Loop Governance

As agents acquire tool-use and execution capabilities, their actions acquire real-world, potentially irreversible consequences. Research on human-in-the-loop (HITL) systems identifies pre-execution confirmation gates as the primary structural mitigation for autonomous agent errors with irreversible consequences.[2] The system described here implements HITL at two levels: the execution gate (all destructive operations) and the canonical entry governance gate (Tier 1 knowledge store modifications).

Destructive scope discovered after CONFIRM → abort + escalate to orchestrator

Figure 5. Execution gate flow. The gate fires before any destructive or side-effecting command. The specialist waits for explicit human CONFIRM before proceeding. Destructive scope detected after CONFIRM triggers immediate abort.

9.1 The Execution Gate

The execution gate applies to all specialist agents in direct mode (called by a human directly, not by the coordinator). In direct mode, a specialist must: present the full execution plan before running any command; include the reversibility rating and blast radius for each step; state explicitly that the human must confirm to proceed; and wait for an affirmative confirmation before executing. Execution does not proceed based on inference about what the human would want — only on an explicit confirmation message.

In background mode (coordinator-delegated), the gate is suppressed because the coordinator has already evaluated the plan and owns the gate. However, a background-mode specialist must still return a scope escalation block if it detects destructive scope beyond what was described in the delegation brief. This preserves human oversight: the coordinator is accountable for what it authorizes, and the specialist is accountable for not exceeding that authorization.

9.2 HITL vs HOTL

The literature distinguishes human-in-the-loop (HITL), where a human must approve before an agent acts, from human-on-the-loop (HOTL), where an agent acts autonomously while a human monitors and can intervene after the fact. The execution gate implements HITL for destructive operations and HOTL for safe and recoverable operations. This hybrid approach reflects the tradeoff between throughput and pre-emptive safety: requiring confirmation for every agent action destroys the value of automation; requiring no confirmation for any action removes the accountability that makes autonomous operation

trustworthy.[11]

9.3 The Canonical Entry Governance Gate

Tier 1 (Canonical) knowledge store entries — safety rules, architectural invariants, non-negotiable constraints — require explicit human confirmation to create or modify. This gate is distinct from the execution gate: it applies to knowledge state, not operational actions. The rationale is asymmetric risk: an incorrect canonical entry actively misleads every agent that queries the knowledge store on any topic the entry covers. The confirmation friction is proportional to this risk. An agent that wants to propose a new canonical rule presents it as a proposal; a human approves and executes the promotion command.

10Session Lifecycle and Knowledge Continuity

A well-designed session lifecycle is the mechanism that prevents knowledge store contents from diverging from actual project state. Two SOPs govern this: the session review SOP (runs at the start of every session) and the documentation SOP (runs after significant changes). The SOPs are the operational layer that keeps the knowledge store current; without them, the tiered retrieval architecture degrades to a static snapshot.

10.1 Session Open

At the start of every session, the following sequence runs:

Step 1: Git synchronization	Pull latest changes. Verify the knowledge store database is current (not an LFS pointer).
Step 2: Knowledge store health check	Run stats query. Verify document and chunk counts are within expected range.
Step 3: Session bootstrap record	Record session open event in the audit database. Capture: branch, last commit, VRAM state, service status.
Step 4: Recent document ingestion	Re-ingest any modified tracked documents using idempotent update semantics. Content-hash comparison ensures unchanged documents are skipped without re-embedding.
Step 5: Context query	Run a context query against the knowledge store for the session's task description. Tier 1 results are always returned first.

10.2 During-Session Recording

During the session, every significant agent output is recorded to the audit database via the event recorder. The recording protocol is dual: the event recorder captures all events (what happened); the knowledge store captures decisions and findings that should persist beyond the session (what we know). Not every event needs to be promoted to the knowledge store — the working memory tier handles session-scoped context automatically. The decision about what to promote to long-lived is made at session close.

10.3 Session Close and Compaction

At session close: record the session close event in the audit database; run the knowledge store garbage collector to expire working memory entries past their TTL; compact the session's working records into a

single distilled long-lived summary; and commit the updated knowledge store database to version control so that it is available in the next session (and in CI environments).

The compaction operation takes all Tier 3 (Working) documents from the current session, concatenates them, re-chunks and re-embeds, and stores the result as a single Tier 2 (Long-lived) distilled note. This prevents unbounded growth of working memory (which would eventually degrade retrieval quality) while preserving the session's knowledge contribution in a queryable form.

10.4 Documentation SOP

When a specialist completes work that modifies tracked items (epic trackers, stub trackers, changelogs), a documentation SOP obligation is triggered. The specialist packages this as a SOP_HANDOFF block to the coordinator; the coordinator executes the SOP before committing. The documentation SOP: identifies which tracked items were closed; updates the relevant tracking documents; generates or updates the changelog entry; and re-ingests the updated documents into the knowledge store. This sequence ensures the knowledge store reflects the current state of the project rather than the state as of the last manual update.

11Implementation Guidance

This section provides practical guidance for implementing the architecture described in this paper. Recommendations are stack-agnostic except where a specific technology is identified as a strong fit for a particular role.

11.1 Minimum Viable System

A minimum viable implementation of this architecture requires: one orchestrator agent; at least one domain specialist; an event recorder with append-only SQLite backing; a knowledge store with at least two tiers (canonical and long-lived) and a basic keyword search; and a session lifecycle SOP that ingests documents at session open. This is sufficient to demonstrate the core properties of the architecture. Add hybrid retrieval, the full tier structure, and the synthesis agent in subsequent iterations.

11.2 Technology Stack Recommendations

Component	Strong fit	Notes
Event recorder backing store	SQLite (WAL mode)	Zero infrastructure, ACID, FTS5 built in, file-level portability
Knowledge store backing store	SQLite (WAL mode) at <10K chunks; pgvector at larger scale	SQLite avoids server process; pgvector supports HNSW indexing
Embedding model	768-dim sentence transformer (local)	nomic-embed-text-v1.5 or equivalent; local avoids API cost and latency
Normalization model	3–7B param quantized LLM, temp=0	Deterministic output required; small model sufficient for structured extraction
Orchestrator / specialist LLM	Best available reasoning model	Claude Sonnet, GPT-4o, Gemini Pro, or local equivalent depending on sensitivity
Agent framework	AutoGen, LangGraph, or custom	Structured output contracts can be implemented in any framework
Version control integration	Git (commit authority to orchestrator only)	Knowledge store DB committed to LFS; session records committed at close

Table 4. Technology recommendations by component.

11.3 Agent Specification Checklist

Every agent in the system should be specified before it is built. A complete agent specification includes the following fields:

- Name and role description
- Domain boundary: exact file types, system layers, or functional scope owned

- Model: which LLM (or local model) backs this agent
- Tools: which tools the agent has access to
- Task modes: which of plan / exec / debug / audit this agent supports
- Output block schemas: exact field definitions for each output block type
- Escalation path: what happens when a request exceeds domain scope
- Audit contract: confirmation that the shared audit contract applies
- Execution gate: whether and how the gate applies in direct vs background mode
- Knowledge base: which KB document IDs are primary references for this agent
- Gothi source tag: the identifier used when recording events from this agent
- Version: spec version number and compatible ecosystem version

11.4 Common Implementation Mistakes

Overlapping domain boundaries. Two agents with authority over the same file type or system layer. Resolves by making boundaries explicit and exclusive. When ambiguous cases arise, assign to the more specific domain and document the decision.

Soft execution gates. Implementing the execution gate as a suggestion rather than a hard stop. The gate must block execution until explicit confirmation arrives in the chat interface, not inferred from context or prior instructions.

Conflating audit and memory. Storing session events in the knowledge store or retrieving context from the audit database. Keep these systems separate. The audit database is append-only and not optimized for semantic retrieval; the knowledge store has TTL semantics that conflict with audit immutability.

Skipping the session SOP. Running the session lifecycle SOP as an optional step. The SOP is how the knowledge store stays current. A team that runs the SOP inconsistently will find that the knowledge store reflects the project state from the last time anyone ran the SOP, not the current state.

Canonical entry proliferation. Promoting too many documents to Tier 1 (Canonical). Canonical entries always surface first and structurally cannot be outranked. A large canonical tier crowds out working context. Reserve Tier 1 for genuine invariants — rules that must apply in all contexts without exception.

12Discussion and Related Work

The architecture described in this paper draws on and extends several active research areas.

12.1 Multi-Agent Coordination Frameworks

AutoGen (Wu et al., 2024) introduced the conversable agent abstraction and demonstrated that hierarchical agent supervision significantly improves performance on complex multi-step tasks compared to single-agent approaches.[4] MetaGPT (Hong et al., 2023) introduced structured communication protocols between specialized agents, showing that SOP-driven workflows reduce coordination errors.[9] ChatDev (Qian et al., 2023) demonstrated the viability of role-specialized agents for software engineering tasks. The architecture described here differs from these frameworks primarily in its emphasis on structural accountability — the audit contract, append-only audit trail, and tiered knowledge governance are first-class architectural components rather than optional additions.

12.2 Retrieval-Augmented Generation

The knowledge store architecture builds directly on the RAG framework established by Lewis et al. (2020).[1] The tiered classification system extends base RAG with priority-weighted retrieval that provides structural guarantees about which content surfaces first. The hybrid dense-sparse retrieval approach is supported by recent empirical work showing significant recall improvements over dense-only retrieval.[8] The use of SQLite as a backing store rather than a dedicated vector database reflects a pragmatic choice for the deployment scale described here: embedded, zero-infrastructure, and Git-compatible.

12.3 Auditability and Traceability

Research on auditable agentic systems has identified runtime event recording and tamper-resistant logging as foundational requirements for accountability in autonomous systems.[7] The append-only audit database with WAL mode SQLite backing is a practical implementation of these requirements. The inline audit contract markers extend this by making reasoning provenance — not just execution traces — visible in the output itself, consistent with the direction identified by Vispute (2026).[10]

12.4 Limitations

Several limitations of the current architecture warrant acknowledgment. First, the coordination bottleneck: a single orchestrator creates a potential throughput ceiling and single point of failure. For systems requiring high parallelism, a distributed coordinator or supervisor hierarchy may be necessary, at the cost of more complex conflict resolution. Second, the linear vector scan in the knowledge store scales to approximately 10,000 document chunks before approximate nearest-neighbor indexing becomes necessary. Third, the audit contract relies on self-reported uncertainty — an agent that does not recognize its own uncertainty will not fire the appropriate markers. Complementary external evaluation mechanisms remain an open research problem.

13Conclusion

This paper has described a four-layer architecture for autonomous development operations that addresses the three primary failure modes of multi-agent systems: context degradation, unverifiable claims, and unconstrained irreversibility.

The key design properties that drive consistent outcomes: domain isolation enforced by hard boundaries and structured escalation paths; structural accountability through the shared audit contract and append-only event recorder; durable memory through tiered hybrid RAG with deterministic priority scoring; and human-gated irreversibility through the pre-execution confirmation gate and canonical entry governance gate.

These properties are mutually reinforcing. Domain isolation makes the audit trail interpretable (events are attributed to known domains). The audit contract makes the knowledge store more reliable (grounding gaps are surfaced rather than confabulated). The knowledge store makes the audit contract more actionable (agents have grounded references to cite). Human-gated irreversibility makes autonomous operation trustworthy because the cost of an error is bounded.

The architecture is not prescriptive about implementation stack. The components described here can be built with a wide range of available frameworks, databases, and language models. What is prescriptive is the structural arrangement: the separation of write and read surfaces in the audit layer; the tiered priority model with deterministic scoring in the knowledge layer; the structured output contracts that make inter-agent communication parseable; and the pre-response reasoning requirement that makes uncertainty visible in the output. These structural properties are what produce reliable outcomes — not the specific technologies used to implement them.

Core insight	Systems that optimize for capability produce impressive output in the short term and degrade unpredictably over time. Systems that optimize for correctness — bounded domains, verifiable claims, durable memory, human-gated irreversibility — can expand capability without sacrificing reliability. The architecture described here is an attempt to get this order of operations right.

Bibliography

1.Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." In *Advances in Neural Information Processing Systems 33*, edited by H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, 9459–9474. Curran Associates, 2020.

2.Shneiderman, Ben. *Human-Centered AI*. Oxford: Oxford University Press, 2022. See also: Mosqueira-Rey, Eduardo, Elena Hernández-Pereira, David Alonso-Ríos, José Bobes-Bascarán, and Ángel Fernández-Leal. "Human-in-the-Loop Machine Learning: A State of the Art." *Artificial Intelligence Review* 56, no. 4 (2023): 3005–3054.

3.Guo, Taicheng, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. "Large Language Model Based Multi-Agents: A Survey of Progress and Challenges." *arXiv preprint arXiv:2402.01680*, 2024. See also: "Multi-Agent Coordination across Diverse Applications: A Survey." *arXiv preprint arXiv:2502.14743*, 2025.

4.Wu, Qingyun, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." In *Proceedings of the International Conference on Learning Representations (ICLR) Workshop on LLM Agents*, 2024.

5.Evans, Eric. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston: Addison-Wesley, 2003. See also: Taibi, Davide, Valentina Lenarduzzi, and Claus Pahl. "The Principle of Least Privilege in Microservices: A Systematic Mapping Study." *Applied Sciences* 16, no. 3 (2026): 1495.

6.Saltzer, Jerome H., and Michael D. Schroeder. "The Protection of Information in Computer Systems." *Proceedings of the IEEE* 63, no. 9 (1975): 1278–1308. See also: OWASP Foundation. "Least Privilege Principle." Accessed May 2026. https://owasp.org/www-community/controls/Least_Privilege_Principle.

7.Kacianka, Severin, and Alexander Pretschner. "Designing Accountable Systems." In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, 424–434. New York: ACM, 2021. See also: "Creating Characteristically Auditable Agentic AI Systems." In *Proceedings of the Intelligent Robotics FAIR 2025*. New York: ACM, 2025.

8.Abdallah, Adel, Bhawani Selvaretnam, and Jamie Twycross. "Hybrid Dense-Sparse Retrieval for High-Recall Information Retrieval." *arXiv preprint arXiv:2501.04695*, 2025. See also: Brown, Andrew, Muhammad Roman, and Barry Devereux. "A Systematic Literature Review of Retrieval-Augmented Generation: Techniques, Metrics, and Challenges." *arXiv preprint arXiv:2508.06401*, 2025.

9.Hong, Sirui, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. "MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework." In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

10. Vispute, Neelmani. "Reasoning Provenance for Autonomous AI Agents: Structured Behavioral Analytics Beyond State Checkpoints and Execution Traces." *arXiv preprint arXiv:2603.21692*, March 2026.

11. Bengio, Yoshua, et al. *International AI Safety Report*. Independent International AI Safety Report, 2025. See also: "The Oversight Game: Learning to Cooperatively Balance an AI Agent's Safety and Autonomy." *arXiv preprint arXiv:2510.26752*, 2025.

This whitepaper describes a general architectural pattern for structured multi-agent systems. It is not specific to any commercial product, vendor, or AI service provider. All design choices described are implementable using open-source tools and publicly available research.